

# Task Manager Client

---

Este projeto consiste numa Aplicação Web Cliente desenvolvida em ReactJS para gestão de tarefas pessoais, que consome a API REST desenvolvida na Parte 1 ([task-manager-api](#)). A aplicação permite autenticação de utilizadores através do fluxo OAuth2 Resource Owner Password Credentials, armazenando o token de acesso JWT no [localStorage](#) do browser, e operações CRUD sobre tarefas e categorias, garantindo que cada utilizador apenas consegue aceder às suas próprias informações.

---

## Tecnologias Utilizadas

---

- ReactJS 18
  - React Router DOM v6 — navegação por URL real
  - Axios — comunicação com a API REST
  - React Context API — gestão de estado global de autenticação
  - Docker / Docker Compose
  - Nginx — servidor web para servir a aplicação em produção
- 

## Estrutura do Projeto

---

```
task-manager-client/  
├── public/  
│   └── index.html  
├── src/  
│   ├── components/  
│   │   ├── routing/  
│   │   │   └── ProtectedRoute.js  
│   │   └── ui/  
│   │       ├── Loading.js  
│   │       └── ErrorMessage.js  
│   ├── constants/  
│   │   └── routes.js  
│   ├── context/  
│   │   └── AuthContext.js  
│   ├── hooks/  
│   │   └── useFetch.js  
│   ├── pages/  
│   │   ├── Home.js  
│   │   ├── LoginPage.js  
│   │   ├── TasksPage.js  
│   │   ├── CategoriesPage.js  
│   │   ├── ProfilePage.js  
│   │   └── NotFound.js  
│   ├── services/  
│   │   └── api.js
```

```
|   |   | App.js  
|   |   | App.css  
|   |   | index.js  
|   |   | index.css  
|   | .dockerignore  
|   | .env  
|   | .gitignore  
|   | Dockerfile  
|   | docker-compose.yml  
|   | nginx.conf  
|   | package.json  
|   | README.md
```

- `components/routing/` → proteção de rotas privadas (`ProtectedRoute`)
- `components/ui/` → componentes reutilizáveis de interface (`Loading`, `ErrorMessage`)
- `constants/` → constantes das rotas da aplicação
- `context/` → estado global de autenticação com React Context API
- `hooks/` → hook personalizado `useFetch` para chamadas à API
- `pages/` → páginas da aplicação
- `services/` → camada de acesso à API REST com Axios

---

## Funcionalidades

---

### Autenticação (OAuth2 — Resource Owner Password Credentials)

- Login com email e password
- Registo de nova conta com auto-login após registo
- Armazenamento do `access_token` em `localStorage`
- Logout com limpeza do token e redirecionamento
- Proteção automática de rotas — redireciona para `/login` se não houver sessão

### Tarefas (CRUD completo)

- Listagem das tarefas do utilizador autenticado
- Estatísticas por estado (pendente / em progresso / feito)
- Filtragem por estado
- Criar, editar e apagar tarefas via modal
- Confirmação antes de apagar

### Categorias (CRUD completo)

- Listagem de todas as categorias
- Criar, editar e apagar categorias
- Visualização com cores distintas por categoria

### Perfil

- Visualizar dados do utilizador autenticado
- Editar nome e email
- Apagar conta com confirmação

---

## Fluxo de Autenticação

---

1. Utilizador introduz email + password no LoginPage
2. Cliente envia POST /oauth/token com grant\_type=password + client\_id + client\_secret
3. API valida credenciais e devolve { access\_token, token\_type, expires\_in }
4. Cliente guarda o token em localStorage
5. Todas as chamadas seguintes incluem: Authorization: Bearer <token>
6. Se a API devolver 401/403, o token é removido e o utilizador é redireccionado para /login

Este fluxo corresponde ao **Resource Owner Password Credentials Grant** do OAuth 2.0 (RFC 6749 §4.3).

---

## Navegação

---

A aplicação utiliza **react-router-dom** v6 com URLs reais:

| URL         | Página          | Autenticação |
|-------------|-----------------|--------------|
| /           | Página inicial  | Pública      |
| /login      | Login / Registo | Pública      |
| /tasks      | Tarefas         | Privada      |
| /categories | Categorias      | Privada      |
| /profile    | Perfil          | Privada      |
| *           | Página 404      | Pública      |

As rotas privadas são protegidas pelo componente **ProtectedRoute**, que redireciona automaticamente para /login se não existir sessão válida.

---

## Docker

---

A aplicação utiliza uma arquitetura **multi-container com 3 imagens** publicadas no Docker Hub, no namespace: **inf26dw2g09**

| Container    | Imagem                         | Descrição                  |
|--------------|--------------------------------|----------------------------|
| mysql_db     | inf26dw2g09/db-task:latest     | Base de dados MySQL 5.7    |
| node_api     | inf26dw2g09/api-task:latest    | API REST Node.js (Parte 1) |
| react_client | inf26dw2g09/client-task:latest | Aplicação Web React        |

O MySQL inclui um `healthcheck` que garante que a API só arranca depois da base de dados estar pronta. A aplicação React é servida pelo Nginx na porta 80.

## Execução

Apenas é necessário o ficheiro `docker-compose.yml` — todas as imagens são puxadas automaticamente do Docker Hub:

```
docker-compose up
```

Para recriar tudo do zero:

```
docker-compose down -v  
docker-compose up
```

URLs disponíveis

| Recurso       | URL                            |
|---------------|--------------------------------|
| Aplicação Web | http://localhost               |
| API REST      | http://localhost:3000          |
| Swagger UI    | http://localhost:3000/api-docs |

Conta de teste

| Campo    | Valor           |
|----------|-----------------|
| Email    | igor1@email.com |
| Password | 123             |

## Variáveis de Ambiente

| Variável          | Valor                 | Descrição            |
|-------------------|-----------------------|----------------------|
| REACT_APP_API_URL | http://localhost:3000 | URL base da API REST |

# Utilização de Ferramentas de IA

---

Durante o desenvolvimento deste projeto foram utilizadas ferramentas de Inteligência Artificial como apoio em partes específicas, nomeadamente:

- na estruturação inicial dos componentes React e da organização das pastas do projeto
- na implementação da camada de serviços com Axios e dos interceptors de autenticação

As restantes componentes do projeto foram desenvolvidas e validadas por mim, recorrendo à IA apenas como apoio complementar sempre que necessário.

---

## Conclusão

---

Este projeto permitiu desenvolver uma Aplicação Web Cliente completa em ReactJS, consumindo uma API REST protegida por OAuth2 com tokens JWT. Foram implementadas operações CRUD sobre tarefas e categorias, navegação por rotas protegidas com `react-router-dom`, e uma arquitetura multi-container com 3 imagens Docker publicadas no Docker Hub, garantindo que a aplicação funciona em qualquer máquina apenas com `docker-compose up`.